

Frontend Engineering Take-Home

Assignment: Hotel Discovery Interface

Overview

As a full-stack engineer at our company, you will help design and build the user interfaces that power travel experiences for millions of customers.

In this assignment, you are tasked with building a lightweight, production-ready frontend application for finding and exploring hotels. Imagine you are building the client-facing discovery platform for a travel app. Users need to be able to efficiently filter through properties, dive deep into a specific hotel's details, and quickly check what types of rooms are open for their travel dates. Your goal is to design a clean, maintainable, and well-structured frontend application that demonstrates your command of component architecture, state management, and user experience principles.

 **Time Commitment:** We value your time and ask that you spend **no more than 3 hours** on this assignment. Focus on the core requirements first—we prefer a clean, working interface over an over-engineered, visually complex but incomplete solution.

Technical Requirements

Your application should implement a clean user interface powered by the provided Mock Data Seed. Make any style choices you see fit.

The interface must deliver three core capabilities:

1. Search & Filter Dashboard

- **Description:** A primary view where users can browse properties.
- **Expectations:** Provide intuitive UI controls (e.g., dropdowns, sliders, or inputs) that allow users to filter the hotel list dynamically by **city**, **star rating**, or **price range**. The list should update smoothly as filters change.

2. Hotel Detail View

- **Description:** A mechanism (e.g., a dedicated route, view toggle, or a modal overlay) to dive deep into a single hotel's information when selected from the dashboard.

- **Expectations:** Render comprehensive data about the property, including its name, address, description, metadata (reviews/ratings), and list of amenities.

3. Room Availability Checker

- **Description:** An interactive component within the Hotel Detail view that lets users see open rooms.
- **Expectations:** Provide an interface for users to select/input a check-in and check-out date. Based on the selected dates, dynamically display which room types are available and show their price per night according to the dataset's `available_dates`.

Note on Input Validation & Edge Cases: All successful layout flows should function smoothly out of the box. Robust error boundary handling or complex form validation is **not** a strict requirement for this assignment. However, if your UI does display errors or empty states (e.g., "No hotels found matching criteria" or "No rooms available for these dates"), they should be cleanly documented in your submission explaining your approach.

Deliverables & Guidelines

- **Language, Frameworks, & Libraries:** You are completely free to select **any language, UI frameworks (React, Vue, Svelte, Vanilla JS), styling libraries, or third-party modules** you want to use. Pick the tools that best showcase your skills.
 - *Note: We use Vue.js and TypeScript on our frontend team, but this is entirely optional for the assignment—use what makes you shine!*
- **AI Tooling Policy:** We heavily welcome and encourage the use of AI tools (e.g., GitHub Copilot, ChatGPT, Claude) to help you build. The only catch is transparency: please include a quick section in your documentation explaining exactly how you leveraged AI during your coding and UI development process.
- **Documentation (`README.md`):** Include instructions on how to install dependencies, run, and test your application locally. Briefly explain your state management approach and component breakdown.
- **Assumptions & Tradeoffs:** Add a separate documentation file that explains any tradeoffs or architectural assumptions you made while writing your code. This is purely to help our engineering reviewers understand your design thinking, not something you will be negatively judged against. Be as detailed as you like, but keep it contained to a single file.
- **Code Quality:** We value readable, scalable code. Prioritize a clear separation of UI elements from data handling logic, component reusability, and rich, useful unit/component tests where appropriate.

Mock Data Seed

To save you time from writing data scripts, use the following master dataset to power your application interface. Save it locally as a JSON file to seed your frontend state. It contains 40 distinct properties distributed across **10 global travel hubs** (4 hotels per city) to ensure your filter logic handles multi-result returns cleanly. Exactly 15% of the inventory is marked with no room availability to test date-boundary layout states.